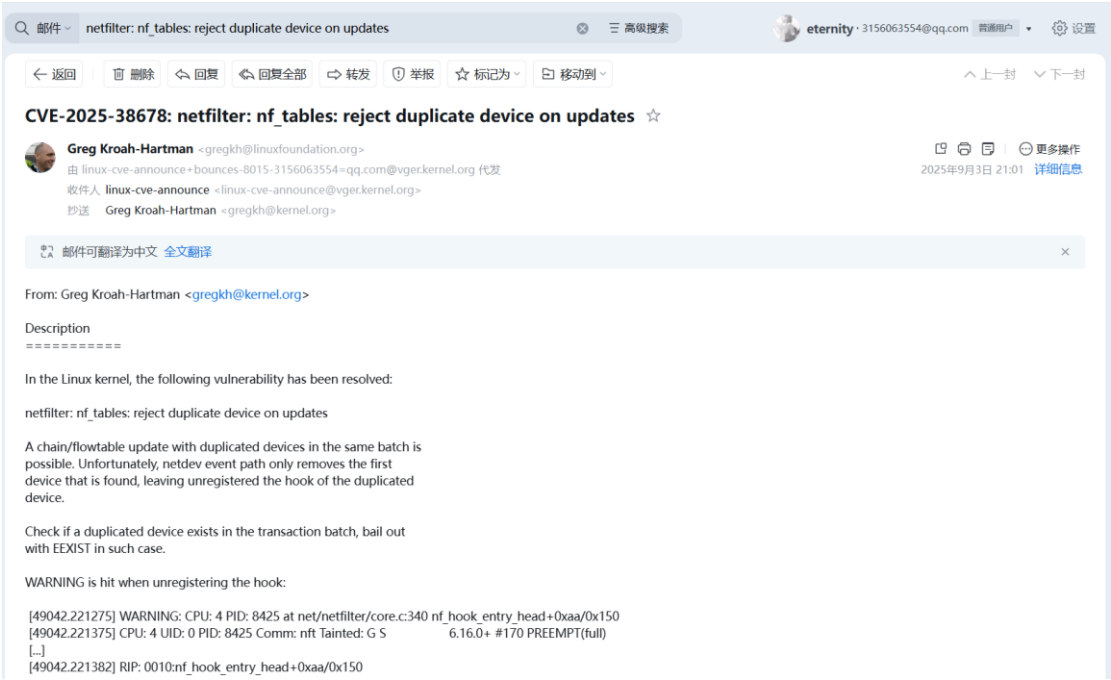


以下源码分析基于 Linux6.15
最近在 kernelctf 中又看到了一个带有 commit 的 Oday，忘了具体是哪个了，这是具体的邮件：



循着邮件给出的 commit_id 简单看一下都做了哪些 patch

```
commit cf5fb87fcd4aaafec55dccc0dc5a9e15ead343973
Author: Pablo Neira Ayuso <pablo@netfilter.org>
Date: Wed Aug 13 02:38:50 2025 +0200

    netfilter: nf_tables: reject duplicate device on updates

    A chain/flowtable update with duplicated devices in the same batch is
    possible. Unfortunately, netdev event path only removes the first
    device that is found, leaving unregistered the hook of the duplicated
    device.

    Check if a duplicated device exists in the transaction batch, bail out
    with EEXIST in such case.

    WARNING is hit when unregistering the hook:

    [49042.221275] WARNING: CPU: 4 PID: 8425 at net/netfilter/core.c:340 nf_hook_entry_head+0xaa/0x150
    [49042.221375] CPU: 4 UID: 0 PID: 8425 Comm: nft Tainted: G S 6.16.0+ #170 PREEMPT(full)
    [...]
    [49042.221382] RIP: 0010:nf_hook_entry_head+0xaa/0x150

    Fixes: 78d9f48f7f44 ("netfilter: nf_tables: add devices to existing flowtable")
    Fixes: b9703ed44ffb ("netfilter: nf_tables: support for adding new devices to an existing netdev chain")
    Signed-off-by: Pablo Neira Ayuso <pablo@netfilter.org>
    Signed-off-by: Florian Westphal <fw@strlen.de>

diff --git a/net/netfilter/nf_tables_api.c b/net/netfilter/nf_tables_api.c
index 13d0ed9d1895..58c5425d61c2 100644
--- a/net/netfilter/nf_tables_api.c
+++ b/net/netfilter/nf_tables_api.c
@@ -2803,6 +2803,7 @@ static int nf_tables_updchain(struct nft_ctx *ctx, u8 genmask, u8 policy,
     struct nft_chain *chain = ctx->chain;
     struct nft_chain_hook hook = {};
     struct nft_stats __percpu *stats = NULL;
+    struct nftables_pernet *nft_net;
+

```

Netfilter 是 Linux 内核比较重要也经常爆出漏洞的一个组件，iptables 的底层支撑，根据描述该 patch 主要防止重复设备，笔者盲猜一手是一个 uaf 漏洞

分析 patch 的代码，主要修改了链和流表的更新逻辑，此时对相关业务代码不熟悉的笔者自然是不太理解到底在 fix 什么的

```

    } else {
@@ -9060,6 +9075,7 @@ static int nft_flowtable_update(struct nft_ctx *ctx, const struct nlmsg_hdr *nlh,
{
    const struct nlattr * const *nla = ctx->nla;
    struct nft_flowtable_hook flowtable_hook;
+   struct nftables_per_net *nft_net;
    struct nft_hook *hook, *next;
    struct nf_hook_ops *ops;
    struct nft_trans *trans;
@@ -9076,6 +9092,20 @@ static int nft_flowtable_update(struct nft_ctx *ctx, const struct nlmsg_hdr *nlh,
    if (nft_hook_list_find(&flowtable->hook_list, hook)) {
        list_del(&hook->list);
        nft_netdev_hook_free(hook);
+       continue;
    }

+   nft_net = nft_per_net(ctx->net);
+   list_for_each_entry(trans, &nft_net->commit_list, list) {
+       if (trans->msg_type != NFT_MSG_NEWFLOWTABLE ||
+           trans->table != ctx->table ||
+           !nft_trans_flowtable_update(trans))
+           continue;

        if (nft_hook_list_find(&nft_trans_flowtable_hooks(trans), hook)) {
            err = -EEXIST;
            goto err_flowtable_update_hook;
        }
    }
}

```

于是笔者使用之前复现 CVE-2022-34918 的 POC 作为 wrapper 简单改一下，变成了创建链和流表（笔者第一个踩的雷，看到 fix 了两处地方误以为是两者结合构成的漏洞）

这里分析流表创建的逻辑可以看到跟设备相关的其实就是这个 hook，当我们传入 hook 的编号优先级以及设备信息，kernel 就会创建 hook 并与设备关联

```

8752     };
8753 };
8754
8755 static int nft_flowtable_parse_hook(const struct nft_ctx *ctx,
8756                                   const struct nlattr * const nla[],
8757                                   struct nft_flowtable_hook *flowtable_hook,
8758                                   struct nft_flowtable *flowtable,
8759                                   struct netlink_ext_ack *extack, bool add)
8760 {
8761     struct nlattr *tb[NFTA_FLOWTABLE_HOOK_MAX + 1];
8762     struct nft_hook *hook;
8763     int hooknum, priority;
8764     int err;
8765
8766     INIT_LIST_HEAD(&flowtable_hook->list);
8767
8768     err = nla_parse_nested_deprecated(tb, NFTA_FLOWTABLE_HOOK_MAX,
8769                                       nla[NFTA_FLOWTABLE_HOOK],
8770                                       nft_flowtable_hook_policy, NULL);
8771
8772     if (err < 0)
8773         return err;
8774
8775     if (add) {
8776         if (!tb[NFTA_FLOWTABLE_HOOK_NUM] ||
8777             !tb[NFTA_FLOWTABLE_HOOK_PRIORITY]) {
8778             NL_SET_BAD_ATTR(extack, nla[NFTA_FLOWTABLE_NAME]);
8779             return -ENOENT;
8780         }
8781     }
8782 }

```

其实 kernel 都在防护 hook 关联同一个设备的问题，此时笔者仍然不知道具体是怎么导致漏洞产生的，但既然防护的如此严谨必然是有些说法的（笔者遇到的第二个坑，此时还以为出现 uaf 的是这个 hook 因为 check 很多）

```

2325 static int nf_tables_parse_netdev_hooks(struct net *net,
2326                                         const struct nlattr *attr,
2327                                         struct list_head *hook_list,
2328                                         struct netlink_ext_ack *extack)
2329 {
2330     struct nft_hook *hook, *next;
2331     const struct nlattr *tmp;
2332     int rem, n = 0, err;
2333
2334     nla_for_each_nested(tmp, attr, rem) {
2335         if (nla_type(tmp) != NFTA_DEVICE_NAME) {
2336             err = -EINVAL;
2337             goto err_hook;
2338         }
2339
2340         hook = nft_netdev_hook_alloc(net, tmp);
2341         if (IS_ERR(hook)) {
2342             NL_SET_BAD_ATTR(extack, tmp);
2343             err = PTR_ERR(hook);
2344             goto err_hook;
2345         }
2346         if (nft_hook_list_find(hook_list, hook)) {
2347             NL_SET_BAD_ATTR(extack, tmp);
2348             kfree(hook);

```

但是疏漏的地方来了，netfilter 对于创建删除这些操作采用的是批处理，不难想到，假使我们先让 hook 绑定一个设备，然后 update 的时候在报文中塞两条一模一样的请求，那么不就绕过 check 成功创建两个绑定同一设备的 hook 了？

事实也的确如此，但 poc 写出来之后竟一时半会想不到验证的点，或者仍不理解说这么做到底有什么用，最后几经周折找到了突破点，这是流表的事件响应函数，就是当设备注销时会向 kernel 发送一个通知，它收到之后判断注销的设备与 hook 链中的 hook 是否是同一个，如果匹配了就将 hook 从 hook 链中解链释放（避免设备注销后仍能通过 hook 去进行操作导致 uaf）

```

9567 static void nft_flowtable_event(unsigned long event, struct net_device *dev,
9568                                struct nft_flowtable *flowtable)
9569 {
9570     struct nft_hook *hook;
9571
9572     list_for_each_entry(hook, &flowtable->hook_list, list) {
9573         if (hook->ops.dev != dev)
9574             continue;
9575
9576         /* flow_offload_netdev_event() cleans up entries for us. */
9577         nft_unregister_flowtable_hook(dev_net(dev), flowtable, hook);
9578         list_del_rcu(&hook->list);
9579         kfree_rcu(hook, rcu);
9580         break;
9581     }
9582 }

```

笔者通过调试最终证实了漏洞的存在（由于在设备注销时只是减少 ref_count 而不是直接释放，笔者干脆在注销之后创建一个 pipe 作为探针方便下断点）

R13+0x18 对应源码的 hook->ops.dev

```

*EIP 0xdead00000000122
*R13 0xffff888100b31480 → 0xffff888100b31360 → 0xffff888100b31f00 → 0xffff888100319040 ← 0xffff888100b31480
*R14 0xffff888100319000 → 0xffff888100319ac8 ← 0xffff888100319000
*R15 0xffff888103827000 ← 0x1010a80a0
*RBP 0xffffc9000067b5e8 → 0xffffc9000067b620 → 0xffffc9000067b638 → 0xffffc9000067b668 → 0xffffc9000067b788 ← ...
*RSP 0xffffc9000067b5a8 → 0xffff888100297170 → 0xffff888102975180 ← 0x4002
*RIP 0xffffffff8236fe1f (nf_tables_flowtable_event+159) ← cmp r15, qword ptr [r13 + 0x18]
CR0 0x80050033 [ PE WP PG ]
*CR3 0x103992000 [virtual: 0xffff888103992000 ← 0x105c77067]
CR4 0x6f0 [ umip fsgsbase smep snap pke cet pks ]
EFER 0xd01 [ NXE ]
GS_BASE 0xffff888235c48000
*FS_BASE 0xfdad380 ← 0xfdad380
CS 0x10 SS 0x18 DS 0x0 ES 0x0 FS 0x0 GS 0x0
[ DISASM / x86-64 / set emulate on ]
> 0xffffffff8236fe1f <nf_tables_flowtable_event+159> cmp r15, qword ptr [r13 + 0x18] 0xffff888103827000 - 0xffff888103827000
0xffffffff8236fe23 <nf_tables_flowtable_event+163> je 0xffffffff8236fe70 <nf_tables_flowtable_event+240>
0xffffffff8236fe25 <nf_tables_flowtable_event+165> mov r13, qword ptr [r13]
0xffffffff8236fe29 <nf_tables_flowtable_event+169> cmp r13, rax
0xffffffff8236fe2c <nf_tables_flowtable_event+172> jne 0xffffffff8236fe1f <nf_tables_flowtable_event+159>

```

而 nft_hook 的头部是个链表头，所以它指向的就是下一个 hook，可见两个 hook 绑定的设备确实是同一个

```

> 0 0xffffffff8236fe1f nf_tables_flowtable_event+159
1 0xffffffff8236fe1f nf_tables_flowtable_event+159
2 0xffffffff813cfd1 notifier_call_chain+97
3 0xffffffff813cb076 raw_notifier_call_chain+22
4 0xffffffff822863e2 call_netdevice_notifiers_info+82
5 0xffffffff82291f46 unregister_netdevice_many_notify+1526
6 0xffffffff82291f46 unregister_netdevice_many_notify+1526
7 0xffffffff82291f46 unregister_netdevice_many_notify+1526
[ THREADS (4 TOTAL) ]
> 1 "" stopped: 0xffffffff8236fe1f <nf_tables_flowtable_event+159>
2 "" stopped: 0xffffffff825cd96b <pv_native_safe_halt+11>
3 "" stopped: 0xffffffff825cd96b <pv_native_safe_halt+11>
4 "" stopped: 0xffffffff825cd96b <pv_native_safe_halt+11>

pwndbg> x/gx 0xffff888100b31480+0x18
0xffff888100b31498: 0xffff888103827000
pwndbg> x/gx 0xffff888100b31360+0x18
0xffff888100b31378: 0xffff888103827000
pwndbg> slab contains 0xffff888103827000
0xffff888103827000 @ kmalloc-cg-4k
Did not finding containing slab.
pwndbg>

```

可见 hook1 确实解链了，但与它绑定同一设备的 hook2 仍在（仍在 hook 链中），并且 net_device 确实已经被释放掉了，但 hook2 仍持有引用，出现了 uaf

```

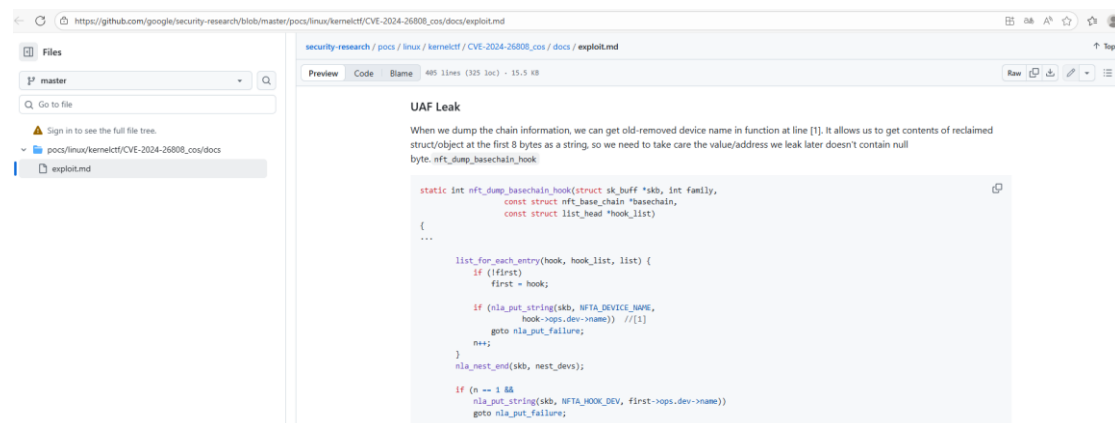
pwndbg> x/gx 0xffff888100b31480
0xffff888100b31480: 0xffff888100b31360
pwndbg>
0xffff888100b31488: 0xdead00000000122
pwndbg> x/gx 0xffff888100b31360
0xffff888100b31360: 0xffff888100b31f00
pwndbg>
0xffff888100b31368: 0xffff888100319040
pwndbg> x/gx 0xffff888100b31360+0x18
0xffff888100b31378: 0xffff888103827000
pwndbg> slab contains 0xffff888103827000
0xffff888103827000 @ kmalloc-cg-4k
slab: 0xffff888103820000 [partial, cpu 0]
status: free
pwndbg>

```

后续笔者想找通过 hook->ops.dev 找到 dev 然后进行读写的代码作为利用点，但苦求无果，最后想到既然防的这么严实那说明有前车之鉴？于是笔者查找资料时还真意外发现了一篇类似的



但比较遗憾只能参考部分，该文章中提到可以利用 `nft_dump_basechain_hook` 泄漏内存



但笔者选用的 6.15 内核中已然换了一种写法

```
1947         goto nla_put_failure;
1948
1949     if (!hook_list)
1950         hook_list = &basechain->hook_list;
1951
1952     list_for_each_entry_rcu(hook, hook_list, list,
1953                             lockdep_commit_lock_is_held(net)) {
1954         if (!first)
1955             first = hook;
1956
1957         if (nla_put(skb, NFTA_DEVICE_NAME,
1958                    hook->ifnamelen, hook->ifname))
1959             goto nla_put_failure;
1960         n++;
1961     }
1962     nla_nest_end(skb, nest_devs);
1963
1964     if (n == 1 &&
1965         nla_put(skb, NFTA_HOOK_DEV,
1966                first->ifnamelen, first->ifname))
1967         goto nla_put_failure;
1968
1969     nla_nest_end(skb, nest);
```

分析发现其实也就在 6.14 时才引进了最后两个成员

```
1200 struct nft_hook {
1201     struct list_head list;
1202     struct nf_hook_ops ops;
1203     struct rcu_head rcu;
1204     char ifname[IFNAMSIZ];
1205     u8 ifnamelen;
1206 };
```

最终，通过堆喷 pipe_buffer 之后 resize 使其分配 4k 页大小的 object 占用释放的 net_device，成功触发 kernel panic

```
test@syzkaller:/ $ uname -a
Linux syzkaller 6.15.0 #3 SMP PREEMPT_DYNAMIC Sun Nov 23 12:00:09 CST 2025 x86_64 GNU/Linux
test@syzkaller:/ $ whoami
test
test@syzkaller:/ $ id
uid=1001(test) gid=1001(test) groups=1001(test)
test@syzkaller:/ $ /exp
veth pair created: vth0 <-> vth1
veth device deleted: vth0
test@syzkaller:/ $ [ 77.976650] -----[ cut here ]-----
[ 77.977441] WARNING: CPU: 1 PID: 46 at net/netfilter/core.c:332 nf_hook_entry_head+0x119/0x170
[ 77.977766] Modules linked in:
[ 77.978412] CPU: 1 UID: 0 PID: 46 Comm: kworker/u16:1 Not tainted 6.15.0 #3 PREEMPT(voluntary)
[ 77.978677] Hardware name: QEMU Ubuntu 24.04 PC (i440FX + PIIX, 1996), BIOS 1.16.3-debian-1.16.3-2 04/01/2014
[ 77.978864] Workqueue: netns cleanup_net
[ 77.979108] RIP: 0010:nf_hook_entry_head+0x119/0x170
[ 77.979241] Code: ff ff 0f 0b 31 c0 5d 31 d2 31 c9 31 f6 31 ff c3 cc cc cc cc 85 d2 75 24 48 85 c9 74 0d 4b 3b b9 08 01 00 00 0f 84 31 ff ff <0f> 0b 31 c0 5d 31 d2 31 c9 31 f6 4
[ 77.979410] RSP: 0010:ffff9000019fcb8 EFLAGS: 00000282
[ 77.979503] RAX: ffff888100313450 RBX: ffff888100b35a90 RCX: ffff88810278f000
[ 77.979566] RDX: 0000000000000000 RSI: 0000000000000005 RDI: ffff88810671e040
[ 77.981867] RBP: ffff8000019fcb8 R08: 0000000000000000 R09: 0000000000000000
[ 77.982553] R10: 0000000000000000 R11: 0000000000000000 R12: 0000000000000005
[ 77.982933] R13: ffff88810671e040 R14: 0000000000000001 R15: ffff888100b35180
[ 77.984425] FS: 0000000000000000(0000) GS:ffff880235c80000(0000) knlGS:0000000000000000
[ 77.986038] CS: 0010 DS: 0000 ES: 0000 CR0: 0000000000000033
[ 77.987136] CR2: 000000003952db08 CR3: 0000000105745000 CR4: 00000000000000f0
[ 77.987660] Call Trace:
[ 77.988664] <TASK>
[ 77.989791] __nf_unregister_net_hook+0x2e/0x200
[ 77.990216] __nf_unregister_net_hook+0x58/0x90
[ 77.990580] __nft_unregister_flowtable_net_hooks+0x4d/0xd0
[ 77.990790] __nft_release_hook+0xca/0x100
[ 77.991257] nf_tables_pre_exit_net+0x72/0xa0
[ 77.991730] cleanup_net+0x1e2/0x410
[ 77.991855] process_one_work+0x191/0x3e0
```

笔者近期较忙，暂无时间研究后续的 exploit，只能给出个 poc 供师傅们研究附上后续 exploit 可能值得参考的 link：

https://github.com/google/security-research/tree/master/pocs/linux/kernelctf/CVE-2024-26808_cos

Email: 3156063554@qq.com